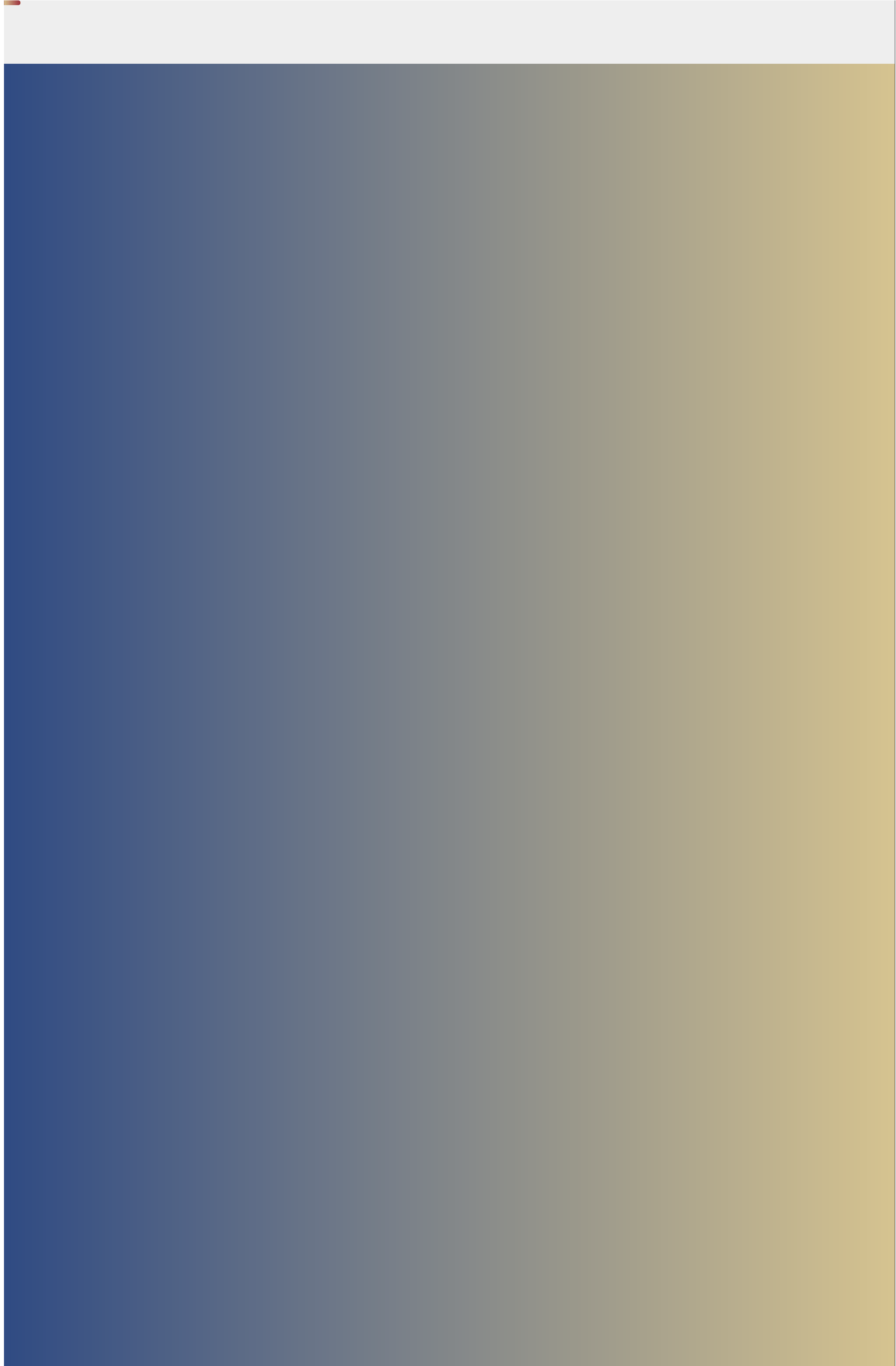






AULA 1

Frameworks para redes neurais e aplicações em visão computacional

O PROFESSOR

Augusto Baffa

Coordenador da graduação em engenharia da computação e professor do Departamento de Informática da PUC-Rio. Mestre em informática pela Unirio e doutor em informática pela PUC-Rio, com especialização MBA em *marketing* pela ESPM e MBA em finanças pela IBMEC.

O PROFESSOR

Luiz Schirmer

Pós-doc no Instituto de Sistemas e Robótica da Universidade de Coimbra, Portugal, e colaborador do laboratório Visgraf do Instituto Nacional de Matemática Pura e Aplicada (IMPA). Atua nas áreas de inteligência artificial e visão computacional. É doutor e mestre em informática pela PUC-Rio e bacharel em ciência da computação pela Universidade Federal de Santa Maria.

O CONVIDADO

Jonatas Grosman

Doutor em informática pela PUC-Rio. Tem mais de 10 anos em experiência no desenvolvimento de projetos de *machine learning*. Recentemente venceu a competição da plataforma Hugging Faces para o processamento de fala e conversão em texto. Alguns de seus projetos são usados em sistemas e experimentos de empresas como Facebook e Hugging Faces.



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Descrever os conceitos básicos de redes neurais (redes neurais de convolução, grafos e *vision transformers*) e suas limitações.
- Desenvolver aplicação de visão computacional no mundo real.
- Analisar os requisitos e as limitações de rede de convolução.

Que história é essa?

Nesta primeira aula, teremos a apresentação dos dois principais *frameworks* para desenvolvimento de redes neurais: **TensorFlow e PyTorch**. Contextualizaremos o uso, as vantagens e as desvantagens de cada um, bem como exemplos práticos de desenvolvimento. Também aprenderemos a desenvolver aplicações para segmentação, classificação e reconhecimento de padrões.

Estudaremos as principais técnicas envolvidas para desenvolvimento de aplicações em visão computacional, como as redes neurais de convolução (CNNs), bem como modelos mais recentes, como as redes de convolução em grafos e arquiteturas baseadas em *transformers*. Do ponto de vista de processamento de linguagem natural, estudaremos as aplicações e o pré-processamento de texto para análise de sentimentos, tokenização e similaridade de documentos. Exploraremos o uso de pacotes Python, como SpaCy e NLTK, para execução dessas tarefas.

TensorFlow X PyTorch

O TensorFlow (TF) e o PyTorch são hoje os dois principais *frameworks* para o desenvolvimento de redes neurais e *machine learning*.

O TensorFlow foi desenvolvido pelo Google Brain, com o objetivo de não apenas acelerar a pesquisa em redes neurais, mas também o *deploy* de aplicação comerciais.

O PyTorch pode ser visto como o “irmão mais novo” do *framework* Torch, que foi desenvolvido baseado na linguagem LUA (criada pela PUC-Rio). Ele foi desenvolvido pela Meta (Facebook); o *framework* Torch foi completamente reescrito e adaptado para parecer nativo da linguagem Python, e não apenas um *wrapper* da sua versão anterior.



A curva de aprendizado para cada um desses *frameworks* pode ser bem diferente e depende da experiência do usuário e de sua

familiaridade com aprendizado de máquina.

Para iniciantes, o TensorFlow pode ser demasiado complicado, visto que apresenta uma complexidade, com implementações em baixo nível para redes neurais. Contudo, desde a versão 2.0, o TensorFlow incorporou oficialmente a API de alto nível Keras, que pode tornar o aprendizado para iniciantes mais “leve”.

Mas atenção: com o uso do Keras, perdemos parte da flexibilidade em customizar arquiteturas de rede neurais e operações de baixo nível que o TensorFlow oferece.

Já o PyTorch tem uma estrutura mais fácil de compreender quando comparado com seu concorrente. A sua própria sintaxe é muito mais próxima da sintaxe tradicional de programação em Python. Ela tem um estilo orientado a objetos, e a sua forma de tratar os dados de treinamento e inferência é mais direta, no estilo de codificação em Python e de tratamento de dados, do que o TensorFlow.

A seguir, você verá algumas diferenças e peculiaridade de cada *framework*.

Grafos dinâmicos e estáticos

Ambos os *frameworks* baseiam seus blocos de execução em grafos, porém há duas diferenças essenciais.

A computação do TF é baseada em grafos, cujos dados de entrada são transformados em tensores (matrizes em altas dimensões), que “fluem” entre os nós computacionais (camadas da rede), daí o nome TensorFlow.

Nas primeiras versões do TensorFlow, o grafo de execução era estático. Isso significa que criamos e conectamos todas as variáveis no início e as inicializamos em uma sessão estática e imutável. Essa sessão de execução e seu grafo eram persistentes, ou seja, não são instanciados novamente após cada iteração do treinamento, tornando-o eficiente.

No entanto, com um grafo estático, os tamanhos das variáveis devem ser definidos no início, o que pode não ser conveniente para algumas aplicações, como, por exemplo, uma aplicação para processamento de linguagem natural cujos dados de entrada podem ser de tamanhos variáveis. Essa abordagem era utilizada em versões anteriores ao TensorFlow 2.0, o que tornava o *framework* pouco flexível, especialmente para a criação de camadas customizadas, e pouco flexível durante o treinamento.

O exemplo a seguir apresenta a estrutura de execução de uma operação de multiplicação utilizando o grafo estático do TensorFlow:

```
cod.py

Importing tensorflow version 1
import tensorflow.compat.v1 as tf

# Initializing placeholder variables of
# the graph
a = tf.placeholder(tf.float32)
b = tf.placeholder(tf.float32)

# Defining the operation
c = tf.multiply(a, b)

# Instantiating a tensorflow session
with tf.Session() as sess:

    # Computing the output of the graph by
    # giving
    # respective input values
    out = sess.run(, feed_dict={a: [3.0], b:
[5.0]})[0][0]

    # Displaying the outputs
```

Diferentemente do TensorFlow, o PyTorch, desde sua criação, usa um grafo dinâmico. Isso significa que o grafo computacional é construído dinamicamente, imediatamente após declararmos as variáveis. Esse grafo é, portanto, reconstruído após cada iteração de treinamento. **Os grafos dinâmicos são flexíveis e nos permitem modificar e inspecionar componentes internos de nossas redes neurais a qualquer momento.**



A principal desvantagem é o tempo de execução do PyTorch por vezes ser maior que o do TensorFlow clássico, visto que temos um tempo adicional para reconstruir o grafo.

Tanto o PyTorch quanto o TensorFlow podem ser mais eficientes dependendo da aplicação e da implementação específicas. Em geral, o PyTorch é mais integrado à linguagem Python e parece mais nativo na maioria das vezes.

Enquanto o TensorFlow poderia ser mais eficiente em termos de tempo de processamento, sua estrutura era demasiado complicada e por vezes afastava novos adeptos ao *framework*. Devido à natureza do PyTorch, muitos novos adeptos o escolhiam devido à curva de aprendizado menor, especialmente para quem trabalha com pesquisa na área.

Embora o TensorFlow tenha a reputação de ser um *framework* focado para aplicações na indústria e o PyTorch tenha a reputação de ser um *framework* focado em pesquisa, isso não é mais verdade.

Muitos dos problemas relacionados ao modo de execução do TensorFlow foram resolvidos na nova versão, e ainda o PyTorch tem investido em aplicações para *deploy* eficiente de modelos para a indústria. Versões mais recentes do TensorFlow removeram o uso de sessões e *placeholders* (métodos para inserir dados de entrada no grafo), e agora também podemos ter a execução de grafos dinâmicos.

Ainda, o TensorFlow incorporou a API de alto nível Keras, que permite a rápida prototipação e teste de modelos.


```
cod.py
#importar tensorflow
import tensorflow as tf
out = tf.math.multiply(3.0,5.0)
```

Debugging

Devido às características do PyTorch, ele permite que usemos ferramentas de *debugging* tradicionais, como pdb ou ipdb.

Este não é o caso do TensorFlow, pois ele tem a opção de usar uma ferramenta especial, chamada tfdbg, que permite avaliar expressões *tensorflow* em tempo de execução. Não é possível utilizar ferramentas de *debugging* tradicionais.

Deploy de modelo em produção

Desenvolver e treinar um modelo de aprendizado profundo é apenas metade do trabalho realizado. Implantar esses modelos de *deep learning* em produção e gerenciá-los é a parte mais difícil.

TensorFlow

O TensorFlow tem uma ferramenta para implantação chamada TensorFlow Serving. Ele é usado para *deploy* de modelos de aprendizado de máquina em servidores gRPC especializados e fornece acesso remoto a eles. A implantação de modelos com o TensorFlow Serving é altamente flexível e se integra perfeitamente à Kubernetes e ao Docker.



O TensorFlow Serving foi projetado para ambientes de produção industrial e é uma boa opção quando o desempenho é uma preocupação. A ferramenta também tem recursos para otimizar o desempenho de modelo e gerenciar custos associados à implantação.

PyTorch

Os usuários do PyTorch têm, por sua vez, o TorchServe. A ferramenta para *deploy* do PyTorch ainda está em estágio experimental, ao contrário do TensorFlow Serving.



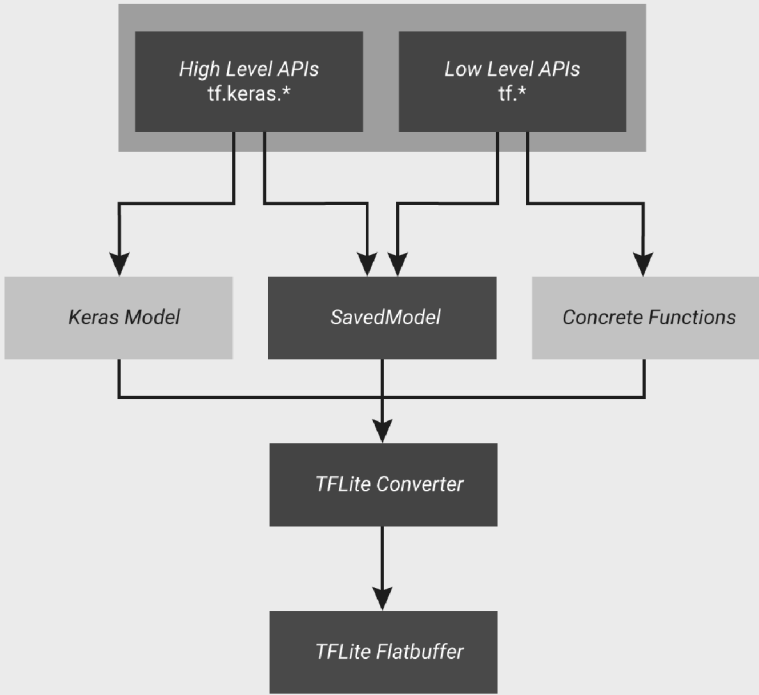
No entanto, o TorchServe tem um conjunto básico de recursos, como um servidor de aplicação, métricas de desempenho, logs de execução e uma API de uso em aplicações *web* (REST).

Deploy em dispositivos móveis

Ambos os *frameworks* nos dão a possibilidade para fazermos *deploy* para aplicações *mobile* e dispositivos embarcados. Contudo, há duas diferenças fundamentais entre eles.

TensorFlow

O TensorFlow tem um *framework* adicional para *deploy* de aplicações *mobile*. O TensorFlow e o TF Lite são *frameworks* completamente separados, portanto é preciso pensar com mais cuidado a respeito de recursos suportados ao treinar o modelo no TensorFlow, pois nem todos os modelos do TensorFlow podem ser convertidos para uso no TF Lite.

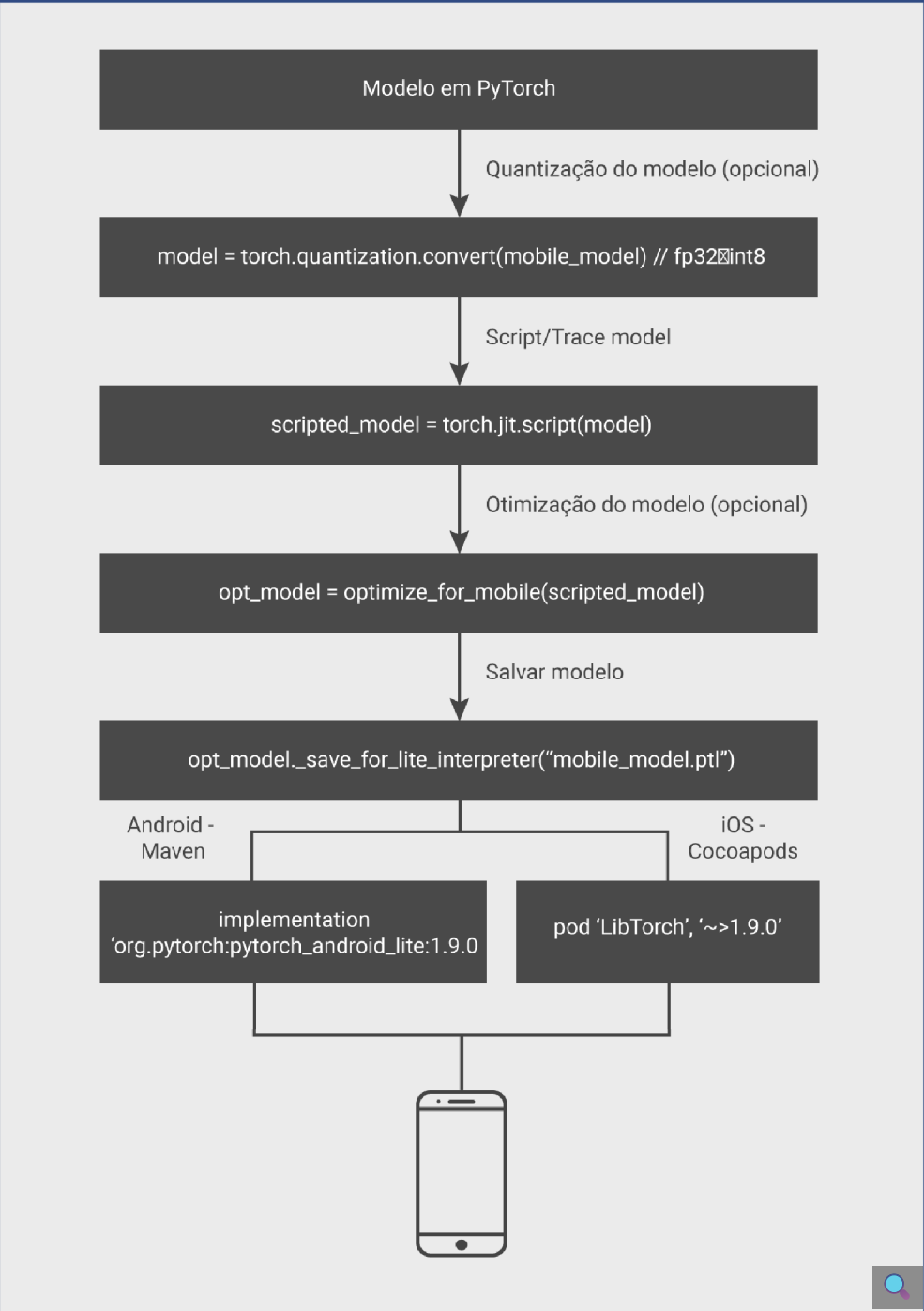


File format Data type Infrastructure



Interativo

Em cima, vê-se uma caixa com fundo preto e letras brancas, com o texto: O TensorFlow Lite mantém uma lista de quais operações são suportadas, que você pode acessar clicando aqui (hiperlink). Abaixo, tem-se o título: PyTorch. O PyTorch tem um workflow mobile que utiliza a mesma base de código que o PyTorch. Uma vez que tenhamos treinado um modelo de rede neural utilizando o PyTorch, basta salvá-lo e carregá-lo utilizando o PyTorch Mobile Lite Interpreter. O PyTorch Mobile é compatível com dispositivos iOS e Android.



Convertendo um modelo de rede neural para TensorFlow Lite

Podemos integrar o TensorFlow Lite para a conversão de modelos com nosso *pipeline* de desenvolvimento em Python para aplicar otimizações, adicionar metadados e muitas outras tarefas que simplificam o processo de conversão.

Interativo

Interativo com o texto: Para converter um modelo do Tensor Flow para TF Lite, basta seguir o exemplo a seguir: Print da aba de um computador com o seguinte código: Linha 1 import tensorflow as tf
Linha 2 cerquilha Convert the model Linha 3 converter igual a tf ponto lite ponto TFLiteConverter ponto from underline saved underline model abre parênteses saved underline model underline dir fecha parênteses cerquilha path to the SavedModel directory Linha 4 tflite underline model igual a converter ponto convert abre parênteses fecha parênteses Linha 5 vazia Linha 6 cerquilha Save the model ponto Linha 7 with open abre parênteses apóstrofo model ponto tflite apóstrofo vírgula apóstrofo wb apóstrofo fecha parênteses as f dois pontos Linha 8 f ponto write abre parênteses tflite underline model fecha parênteses. Na segunda página do interativo, vemos o texto:O TensorFlow ainda nos permite utilizá-lo para execução em código Python. Isso pode ser útil ao prototiparmos aplicações para dispositivos embarcados que permitam a execução de código Python. Para executar um modelo TF Lite em código Python, precisamos instanciar o interpretador TF Lite e passar o path do modelo salvo. Confira no exemplo: Print da aba de um computador com o seguinte código: Linha 1 interpreter igual a tf ponto lite ponto Interpreter abre parênteses model underline path igual a model underline path fecha parênteses Linha 2 interpreter ponto allocate underline tensors abre parênteses fecha parênteses Linha 3 vazia Linha 4 cerquilha Get input and output tensors ponto Linha 5 input underline details igual a interpreter ponto get underline input underline details abre parênteses fecha parênteses Linha 6 output underline details igual a interpreter ponto get underline output underline details abre parênteses fecha parênteses Linha 7 interpreter ponto set underline tensor abre parênteses input underline details abre colchetes 0 fecha colchetes abre colchetes apóstrofo index apóstrofo fecha colchetes vírgula abre colchetes input underline data fecha colchetes fecha parênteses Linha 8 interpreter ponto invoke abre parênteses fecha parênteses Linha 9 cerquilha The function apóstrofo get underline tensor abre parênteses fecha parênteses apóstrofo returns a copy of the tensor data ponto Linha 10 cerquilha Use apóstrofo tensor abre parênteses fecha parênteses apóstrofo in order to get a pointer to the tensor ponto Linha 11 output underline data igual a interpreter ponto get underline tensor abre parênteses output underline details abre colchetes 0 fecha colchetes abre colchetes apóstrofo index apóstrofo fecha colchetes fecha parênteses.

Qual *framework* escolher?

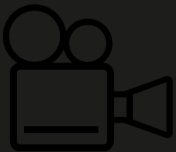
Não há uma resposta definitiva para tal questão. Tudo depende das suas experiências e dos seus objetivos.



Fique ligado

Se você é um iniciante no meio, o PyTorch pode ser a escolha mais amigável. Se você busca eficiência para aplicação em larga escala de ML ou uma carreira voltada à indústria, o TensorFlow pode ser uma melhor opção, dado seu suporte e ferramentas disponíveis para aplicações industriais. Contudo, uma base sólida com relação aos conceitos da ciência de dados e *machine learning* é crucial para o sucesso de qualquer profissional na área, independentemente do *framework* escolhido.

Dado que hoje ambos os *frameworks* tendem a ter mais semelhanças em suas versões mais recentes, o profissional que escolher qualquer um dos *frameworks* como base de aprendizado não terá tantas dificuldades em fazer transições de um para o outro. De todo modo, a recomendação principal é explorar ambos os *frameworks* e escolher aquele com o qual você se sente mais confortável e que é mais adequado ao seu negócio.



Diferenças e semelhanças entre PyTorch e TensorFlow

Neste vídeo, você aprenderá as diferenças de implementação entre TensorFlow e PyTorch, reforçando os conceitos de grafo estático e dinâmico e estudando as diferenças no código de implementação de uma rede de convolução simples em PyTorch e TensorFlow . Além disso, vai aprender a utilizar a ferramenta Tensorboard para analisar o treinamento de sua rede neural.

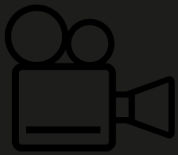


Aplicações de Visão Computacional – Redes Neurais de Convolução

Classificação e detecção de objetos é uma das principais e mais clássicas tarefas em visão computacional.

Classificação e detecção de objetos

Veja mais sobre o assunto no vídeo a seguir.



Aplicações básicas de CNN: detecção e classificação de objetos

Normalmente, o objetivo da classificação é, dado um conjunto de fotografias ou vídeos, identificar a posição de objetos nessas imagens e classificá-los conforme um conjunto de informações anotadas. Acompanhe o vídeo para entender como tudo isso funciona.



Assim, dois conceitos principais se fazem necessários: diferenciar classificação e localização.

A classificação de imagens envolve a prevenção da classe de um objeto em uma imagem. A localização de objetos refere-se a identificar a localização de um ou mais objetos em uma imagem e desenhar uma *bouding box* em torno do objeto. A detecção de objetos combina essas duas tarefas, localizando-as e classificando um ou mais objetos em uma imagem.

Portanto, podemos dividir esses conceitos em três tarefas principais. Clique nas setas e acompanhe.

Interativo

Interativo com o texto: Classificação de Objetos: Identificar a classe de um objeto em uma imagem. Normalmente a imagem contém apenas um objeto a ser classificado. Aqui, vê-se a imagem de um gato com pelagem longa, apresentando uma mistura de cores marrom, preto e branco, de olhos verdes. Ele está cercado por folhagens e flores. A expressão é atenta, como se estivesse fixado em algo fora do quadro da imagem. Nela, acima, vê-se: Classificação e, abaixo, Gato. No segundo slide do interativo, vemos o texto: Localização: Identificar se existe um objeto de interesse em uma imagem. Podemos receber uma imagem com um ou mais objetos, e após identificar a posição de cada um, desenhamos uma bounding box sobre a área de interesse. Aqui, vê-se a imagem de um cão e um gato ao ar livre, ambos posicionados na grama. O cão, à direita, está com um retângulo azul em volta e parece ser da raça Corgi, de corpo compacto, pernas curtas e expressão alerta e feliz. À esquerda, o gato está com um retângulo vermelho em volta, apresentando pelo longo e uma expressão calma e atenta. Eles estão em um cenário ensolarado com a grama verde bem visível. No terceiro slide do interativo, vemos o seguinte texto: Detecção de objetos: identifica a presença de objetos e os classifica um a um. Combina os dois conceitos anteriores. Aqui, vemos a mesma foto anterior, porém dessa vez, abaixo da foto, vemos ‘gato’ escrito em vermelho – a mesma cor do retângulo em sua volta, e ‘cachorro’ escrito em azul – a mesma cor do seu retângulo.

Nas próximas seções, apresentamos dois modelos que podem ser usados em detecção e classificação de objetos: **YOLO e MobileNet**.

YOLO

You only look once (YOLO) é um dos sistemas para detecção de objetos em tempo real considerados estado da arte. Pode ser executado mesmo em dispositivos móveis e tem um desempenho razoável, obtendo uma taxa de atualização superior a 30 *frames* por segundo quando analisando vídeos.



Muitos sistemas mais tradicionais, como a R-CNN, recalculam a classificação e a localização de objetos em um único *frame*, isto é,

aplicam o modelo a uma imagem em vários locais e escalas. As regiões de alta probabilidade nas imagens são consideradas detecções.

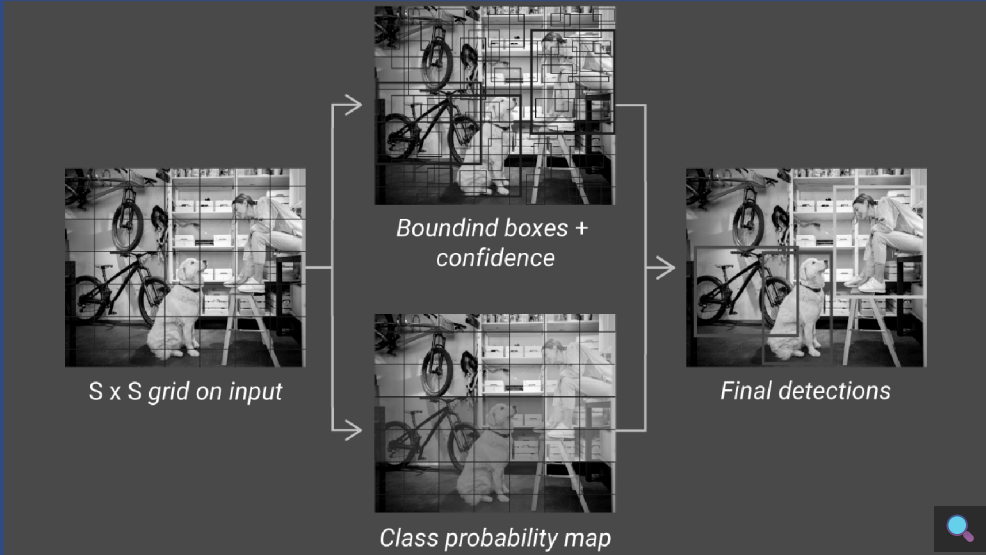
Já o YOLO realiza a detecção uma única vez sobre a imagem como um todo. Essa rede divide a imagem em regiões e prevê *bouding boxes* e probabilidades para cada região em paralelo.

- O sistema divide a imagem de entrada em uma *grid* de tamanho $S \times S$. Se o centro de um objeto estiver em uma célula da *grid*, essa célula é responsável por detectar esse objeto.
- Cada célula da *grid* define pelo menos uma *bounding box* B e um *score* associado a ela, o qual define se foi de fato detectado um objeto naquela região. Esse *score* de confiança reflete o quão confiante o modelo está de que a *bounding box* contém um objeto.
- Se não há um objeto dentro dessa *bounding box*, o *score* de confiança é zero; caso contrário, queremos maximizar o valor em que esse *score* se aproxima à IOU (intersection over *union*) entre a *bounding box* prevista e o dado anotado no treinamento.
- Cada *bounding box* contém cinco atributos: coordenadas x, y, a largura e altura e o seu *score* associado. As coordenadas (x, y) representam o centro da *bounding box* em relação aos limites da célula da *grid*. A largura e a altura são previstas em relação a toda a imagem. Finalmente, o *score* de confiança representa a IOU entre a *bounding box* e o nosso *ground truth*.

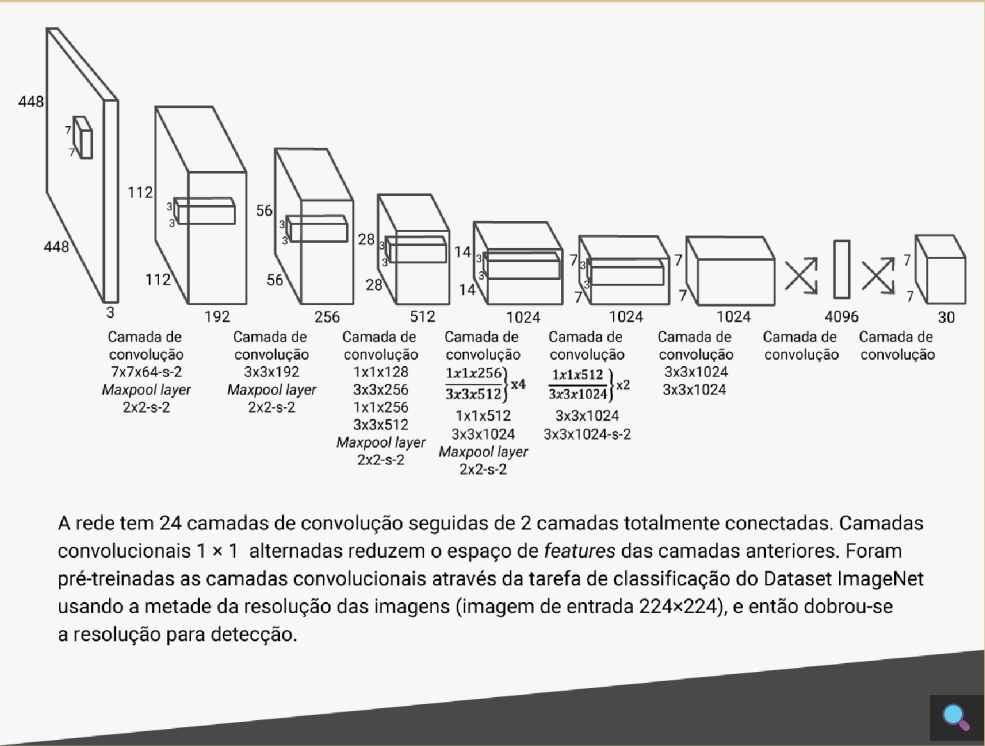
O modelo de rede neural também tem como *output* probabilidades de uma classe de objeto. Essas probabilidades são condicionadas a cada célula da *grid*, contendo um objeto. Ao final, multiplicamos a probabilidade da classe e as previsões do *score* de objetos das *bounding boxes*, ou seja:

$$\text{pred} = \text{Pr}(\text{Class}|\text{Object}) * \text{Pr}(\text{Object}) * \text{IOU}_{\text{verdade}}$$
$$\text{pred} = \text{Pr}(\text{Class}) * \text{IO}$$

O que nos dá o grau de confiança para a classe detectada na *bounding box*. Ao final, todas as informações são associadas para gerar a detecção. A imagem resume as duas saídas do modelo.



Na figura a seguir, você verá que a arquitetura da rede neural do YOLO é baseada na *GoogleLeNet*, com 24 camadas de convolução seguidas por 2 camadas totalmente conectadas.



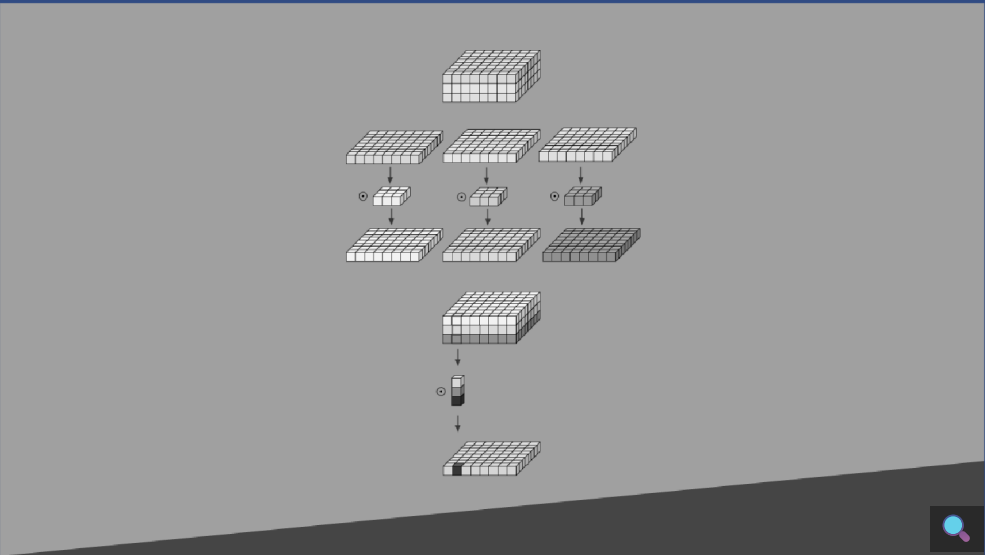
MobileNet

Como o próprio nome já indica, a **MobileNet** é um modelo de rede neural eficiente cujo objetivo é ser usado em dispositivos móveis. Essa rede tem muito menos parâmetros do que um modelo tradicional, como VGG ou LeNet, e, ainda, devido à sua arquitetura, demanda muito menos poder de processamento.

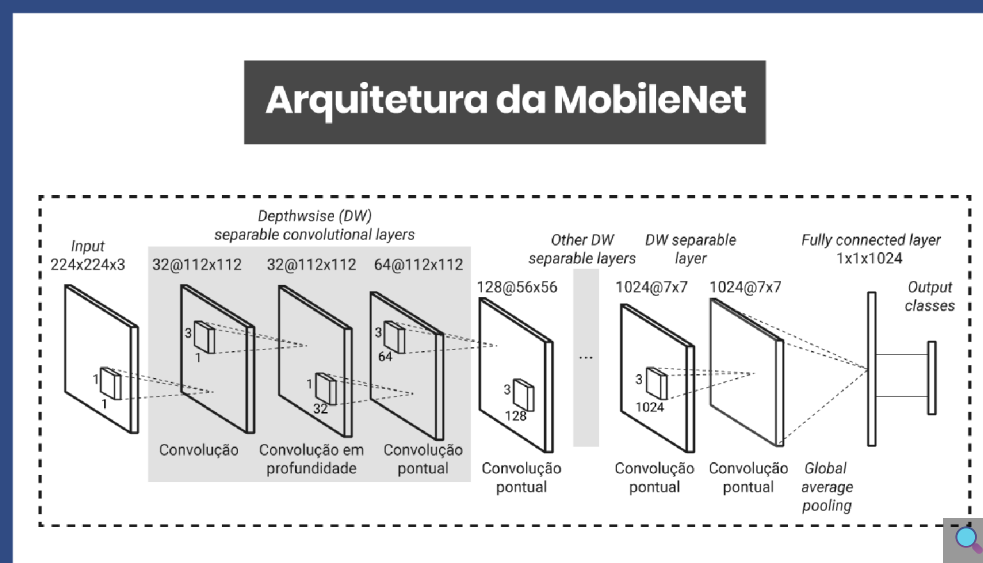
A MobileNet é baseada em *depthwise separable convolutions*, que lidam não apenas com as características espaciais da imagem, mas também com a profundidade, ou seja, os canais RGB.

Nesse processo, um kernel de convolução é separado em outros dois para realizar duas convoluções: uma em profundidade e uma ponto a ponto. Essencialmente, a convolução em profundidade (*depthwise*) aplica um único filtro para cada canal de entrada, e a convolução pontual (*pointwise*) aplica uma convolução de 1 x 1 kernel para combinar as saídas. Esse processo reduz de 8 a 9 vezes o custo computacional necessário para uma operação de convolução.

Veja um exemplo dessa operação:



A arquitetura básica da MobileNet pode ser vista na figura:



Para detecção de objetos, a MobileNet utiliza uma arquitetura chamada SSD (*Single Shot Detection*). Ao usar a SSD, precisamos de apenas uma única imagem para detectar vários objetos e não particionar a imagem em múltiplas partes menores, enquanto abordagens clássicas, como a R-CNN, precisam de duas imagens pelo menos, uma para gerar probabilidades de uma região conter objetos e outra para detectar o objeto de cada região proposta.



Aprenda mais

Para mais informações a respeito da SSD, [clique aqui](#) e consulte o documento.

Em resumo, MobileNets são modelos pequenos, de baixa latência e baixo consumo de energia parametrizados para atender às restrições de recursos de uma variedade de dispositivos.



Aprenda mais

A MobileNet está disponível para download e uso em código Python através do TensorFlow Hub, que você pode acessar clicando [aqui](#).

A seguir, vamos aprender mais sobre a segmentação de objetos.

Segmentação de objetos

Segmentação de objetos é o nome dado à categorização de cada pixel de uma imagem atribuída a uma classe em particular. Imagine como o exemplo clássico a análise de imagem de raios X. Podemos treinar uma rede neural de modo que atribua pixels da imagem em regiões onde possa haver uma possível anomalia. Isso pode ajudar especialistas no domínio a validar mais facilmente possíveis problemas encontrados. Nesse ponto, podemos pensar na semelhança com detecção de objeto, porém há diferenças fundamentais.

Ao aplicarmos modelos de detecção de objetos, só geramos uma *bounding box* correspondente a cada classe de objetos encontrados na imagem, porém não temos informação sobre a forma dos objetos. A segmentação de imagem criará máscaras para cada objeto,

portanto, será útil entender detalhes granulares sobre a forma detectada.

Há dois tipos principais de segmentação de objetos:

Interativo

Interativo com o seguinte texto: Segmentação semântica: Este é o processo de atribuir uma label a cada pixel da imagem. A segmentação semântica trata cada objeto de uma mesma classe de maneira conjunta, sem os distinguir, e vários objetos de uma mesma classe são tratados como uma entidade única. À direita do texto, uma imagem mostra um exemplo de reconhecimento de imagem e segmentação semântica. Na parte superior, temos uma foto de três ovelhas e um cachorro, com uma caixa delimitadora vermelha ao redor do cachorro, indicando que o sistema de reconhecimento de imagem identificou-o com uma certa probabilidade abre parênteses P 0.6 para ovelha, P 0.3 para cachorro, P 0.1 para gato e P 0.0 para cavalo fecha parênteses. As ovelhas são cercadas por uma caixa azul. Na parte inferior, a mesma cena é apresentada, mas com a segmentação semântica aplicada: as ovelhas são cobertas por silhuetas azuis e uma área vermelha indica onde o sistema identificou outra entidade, o cachorro. No segundo slide, vemos o texto: Segmentação por instâncias: A segmentação por instância trata vários objetos da mesma classe como objetos individuais. Normalmente, a segmentação por instâncias é mais difícil do que a segmentação semântica. À direita do texto, vemos a mesma imagem anterior, dessa vez com detecção de objetos, onde cachorro está demarcado por uma caixa vermelha, enquanto as ovelhas são demarcadas por caixas em diferentes tons de azul. Na segmentação por instâncias, abaixo, as ovelhas são cobertas por silhuetas de diferentes tons de azul e o cachorro é coberto por uma silhueta vermelha.

Casos de uso no mundo real

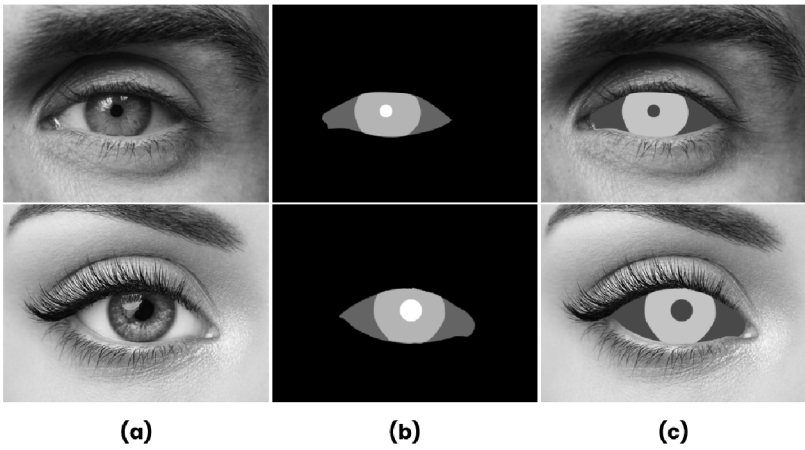
Acompanhe alguns exemplos reais no infográfico a seguir:



Segmentação de objetos no mundo real

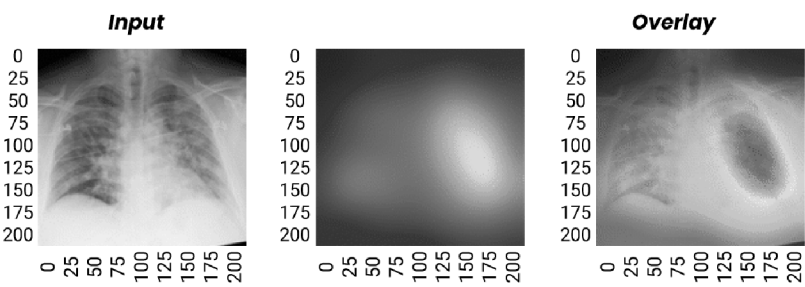
Segmentação Facial

Usada em reconhecimento de expressões faciais, *liveness detection* em sistemas biométricos e previsão de gênero da etnia dos indivíduos. A segmentação semântica permite essas tarefas separando regiões do rosto em atributos importantes, como boca, queixo, nariz, olhos e cabelo.



Imagens médicas

A segmentação de imagens é usada em diagnósticos médicos, especialmente na avaliação de análises de raios X e exames de ressonância magnética.



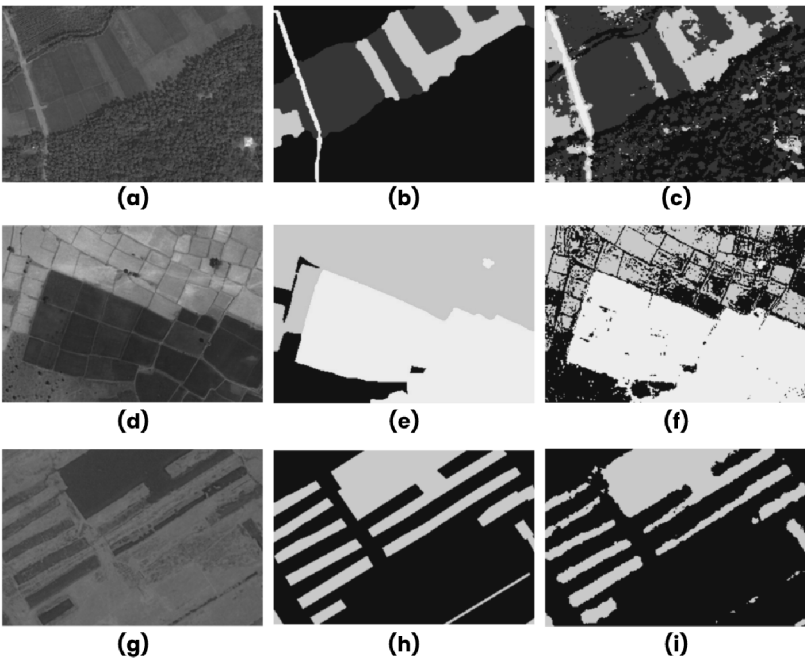
Self-driving

A direção autônoma é uma tarefa extremamente complexa que necessita de desempenho em tempo real. A segmentação semântica é usada para identificar objetos como outros carros e sinais de trânsito e regiões como pistas e calçadas.



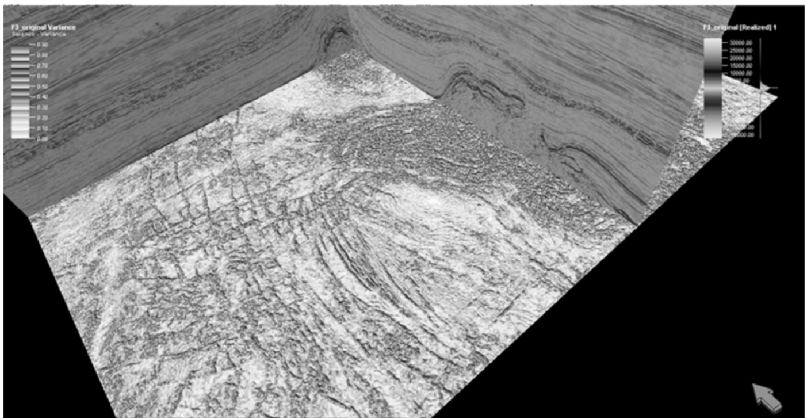
Processamento de imagens de satélite

Imagens aéreas ou de satélite cobrem uma grande extensão de área terrestre, e modelos de IA podem ser usados na agricultura e em sensoriamento remoto.



Processamento de imagens sísmicas

Muitas aplicações em geofísica e na indústria do petróleo também fazem uso de modelos de segmentação. Normalmente, na geofísica, a aplicação direta é na detecção de falhas geológicas.

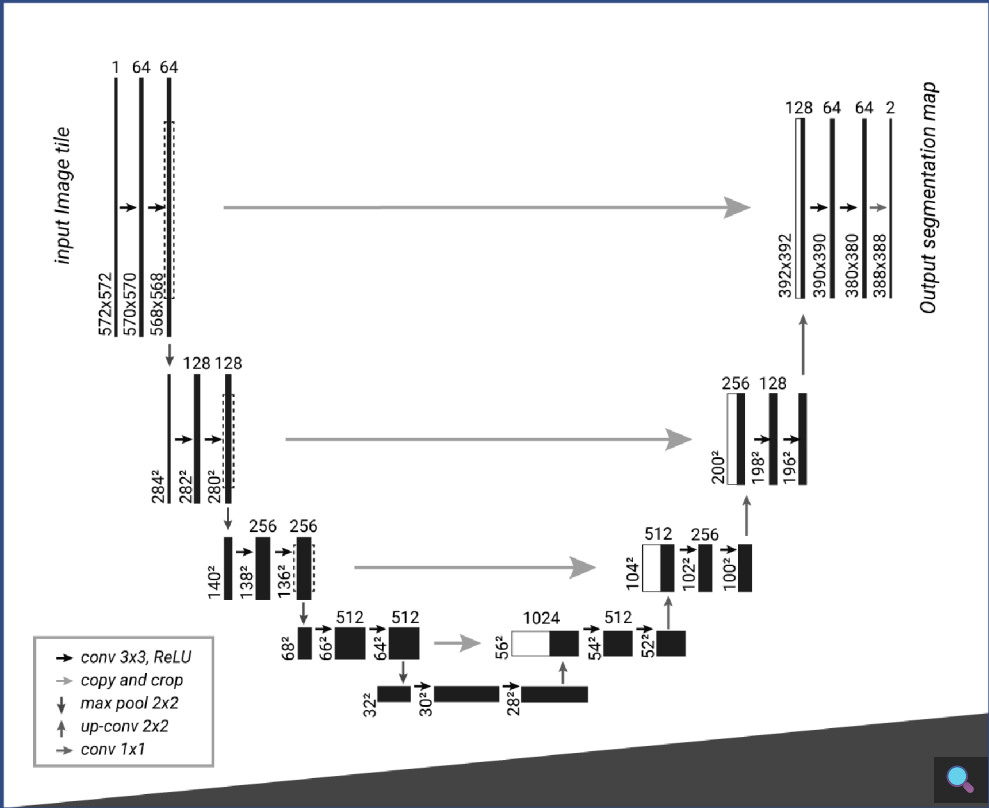




Redes de convolução para segmentação: UNET

A rede neural de convolução UNET é uma rede totalmente convolucional, isto é, não apresenta em sua saída camadas totalmente conectadas. Normalmente, a sua saída nos dá um mapa de “calor” para a posição de cada objeto detectado na imagem, e após calculamos de fato a *label* de cada pixel. A UNET é chamada assim devido à sua arquitetura, realizando sucessivas operações de convolução em duas partes.

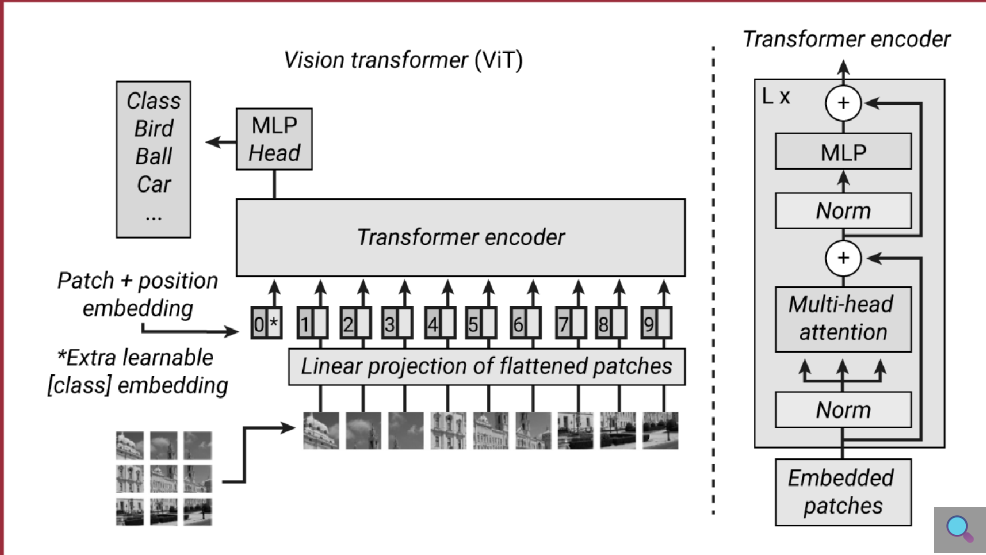
- A primeira etapa é chamada de *encoder* (também chamado de “contração”), que é usada para capturar o contexto na imagem. O *encoder* é apenas uma pilha tradicional de camadas de convolução seguidas de *maxpooling*.
- A segunda etapa é chamada de *decoder* (também chamada de “camada de expansão”), que é usada para permitir a localização precisa usando convoluções transpostas. Ela é uma das arquiteturas mais usadas para segmentação semântica, em especial para análise de imagens médicas.



Vision transformers

Apesar de as redes de convolução terem se estabelecido em muitos casos com o principal *building block* para tarefas de classificação de imagens, uma nova arquitetura emergiu recentemente como o estado da arte: *vision transformers*. A ideia básica do *vision transformer* é uma generalização para imagens de seu irmão mais velho, o clássico *transformer* para análise de texto. A ideia é basicamente decompor as imagens de entrada como uma série de *patches*, que, uma vez transformados em vetores, estes são vistos como *tokens* por um bloco clássico do *transformer*.

Um exemplo dessa arquitetura pode ser visto na imagem a seguir.



O *vision transformer* recentemente conseguiu superar em alguns casos a *performance* das redes de convolução para classificação de objetos. Contudo, essa arquitetura é muito mais difícil de treinar, e seu desempenho depende muito dos hiperparâmetros escolhidos, do conjunto de dados e da profundidade da rede.

Em tarefas de classificação de imagens, muitos trabalhos têm combinado esse modelo com convoluções tradicionais a fim de explorar as vantagens de ambas as técnicas em conjunto. Nessa ideia, uma camada de CNNs gera um mapa de *features* a partir de imagens de entrada. Após, um *tokenizer* traduz esse mapa de *features* para uma série de *tokens*, que são, então, enviados ao *transformer*, que aplica o mecanismo de atenção para produzir uma série de *tokens* de saída.

Ao final, uma nova projeção é feita, em que os *tokens* são reconectados para gerar um novo mapa de *features* atualizado. Junto com o *transformer*, esse processo permite que exploremos globais da imagem e identifiquemos mais facilmente informações contextuais.

Nesta aula, você aprendeu os conceitos básicos das redes neurais e suas limitações, a aplicação da rede neural no mundo real com processamento de linguagem natural e como fazer a análise dos requisitos e das limitações do *deploy* de redes neurais.

O passo seguinte é você partir para uma discussão social e prática com o olhar crítico.



Reflita

Agora que você conheceu mais um dos desafios encontrados no treinamento e na aplicação de rede neurais de convolução, chegou a hora de exercitar o cérebro e refletir sobre como a qualidade dos dados de treinamento usados impactam a *performance* de nosso modelo.

Com base no que estudamos nesta aula, em um primeiro momento, reflita: como podemos determinar de fato se os resultados de uma rede neural podem ser confiáveis?

Em um segundo momento, você irá refletir sobre: desde a revolução estabelecida pelo nascimento do aprendizado profundo em 2012, as redes de convolução se estabeleceram como a principal opção para resolução de problemas em visão computacional. No entanto, em 2021, um concorrente à altura surgiu para desbancar as CNNs: os *vision transformers*.

Dados os desafios e os conceitos apreendidos durante o curso, e de acordo com sua experiência, você acredita que esse novo tipo de arquitetura pode representar uma revolução no meio?



Referências

Clique aqui para acessar as referências e créditos desta aula.



CAELLES, S. *et al.* One-shot video object segmentation. 2016. *In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR)*, 2017, Honolulu. **Proceedings** [...]. Honolulu: Hawaii Convention Center, 2017. p. 221-230.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. London: MIT Press, 2016.

HEATON, J. Ian Goodfellow, Yoshua Bengio, and Aaron Courville: deep learning. **Genetic Programming and Evolvable Machines**, v. 19, n. 1–2, p. 305–307, 2018.

HOWARD, A. G. *et al.* MobileNets: efficient convolutional neural networks for mobile vision applications. **arXiv [cs.CV]**, 2017.

HUANG, H. *et al.* UNet 3+: a full-scale connected UNet for medical image segmentation. *In: 2020 IEEE INTERNATIONAL CONFERENCE ON ACOUSTICS, SPEECH AND SIGNAL PROCESSING (ICASSP)*, 2020, Barcelona. **Proceedings** [...]. Barcelona, 2020.

JOSEPH, F. J. J.; NONSIRI, S.; MONSAKUL, A. Keras and TensorFlow: a hands-on experience. *In: PRAKASH, K. B. Advanced deep learning for engineers and scientists: a practical approach*. Cham: Springer International Publishing, 2021. p. 85–111.

KETKAR, N.; MOOLAYIL, J. Introduction to PyTorch. *In: CHOLLET, F. Deep learning with Python*. Berkeley, CA: Manning Publications, 2021. p. 27–91.

LI, Y. *et al.* **The secrets of salient object segmentation**. *In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION*, 2014, Columbus. **Proceedings** [...]. Columbus, 2014.

SHARIR, G. *et al.* An image is worth 16x16 words, what is a video worth? **DeepAI**, 2021. Disponível em: <https://deepai.org/publication/an-image-is-worth-16x16-words-what-is-a-video-worth>. Acesso em: 25 jan. 2023.

Banco de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock.

